

Formal Verification of Lightning-Network Multi-Hop HTLC Routing in Tamarin 1.8

A modular, machine-checked model with security analysis and
an empirical study of the prover’s tractability boundaries

(author)

August 24, 2026

Abstract

We formalise Lightning-Network multi-hop HTLC (Hash Time-Locked Contract) routing in the Tamarin symbolic prover (v1.8) under a full Dolev–Yao adversary. The protocol is captured as a *modular package of five theories* totalling **81 machine-checked lemmas**, covering the two-party channel lifecycle, multi-hop routing with fees, an N -hop generalisation, staggered-CLTV timing safety, and the necessity of the underlying liveness assumption. Beyond re-establishing the standard safety properties (payment atomicity, intermediary safety, value conservation), we (i) harden the model against a forwarding-replay flaw by replacing a global axiom with a structural linear-resource guard; (ii) machine-check the known *wormhole* fee-theft attack, both as a reachability result and quantified to the exact stolen fees; (iii) prove economic-soundness theorems (end-to-end conservation, strictly positive fees, no value creation) and expose a grieving DoS; and (iv) establish *assumption minimality* empirically—removing the intermediary-claim restriction yields a concrete counterexample. A recurring theme is *tractability*: we pin down the precise combination of ingredients (signing + natural-number induction + an unbounded block clock) that exhausts the prover, and map two further boundaries—signed-message N -hop routing and reusable channels—where automated search stops scaling. Every positive and negative result was executed, not assumed.

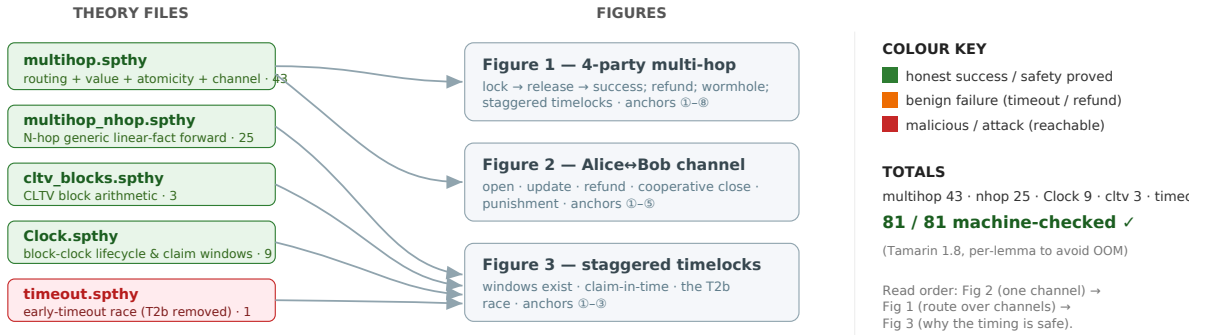
Contents

1	Introduction and scope	1
2	Architecture: a modular five-theory package	2
3	Methodology	2
4	Results: the core model	3
4.1	Multi-hop routing (<code>multihop.spthy</code>)	3
4.2	Timing safety (<code>Cltv</code> , <code>Clock</code> , <code>timeout</code>)	3
4.3	Statistics	4
5	Security analysis	4
5.1	Forwarding-replay hardening (axiom $\rightarrow$ structure)	4
5.2	Wormhole attack (reachable and quantified)	4
5.3	Economic soundness and grieving	5
5.4	Assumption minimality (T3 is load-bearing)	6

6	Generalisation to N hops	6
7	What failed, and the tractability boundaries	7
8	Reproduction	8
9	Conclusion	9
A	Figure index	9

Verification map — 5 theories · 4 figures · 81 lemmas

where each file's lemmas live, and how the figures fit together



multihop_nhop.spthy is the N-hop generalisation (25 lemmas, ~103.5s); the reusable-channel boundary spike is documented separately. See nhop_soundness for the refinement theorem transferring safety to all N.

Rough draft — master index. Open here, then drill into Fig 1 / 2 / 3.

Figure 1: Verification map: five theory files, four figures, 81 lemmas. Green = honest success / safety proved; amber = benign failure (timeout / refund); red = malicious / attack shown reachable.

1 Introduction and scope

Lightning [1] is a Layer-2 payment-channel network: parties transact off-chain and use the blockchain only to settle disputes. A multi-hop payment routes value along a path Alice → Bob → Carol → Dave by chaining HTLCs that share one payment hash $y = H(x)$; the receiver reveals the preimage x , which propagates back along the path, redeeming each HTLC. Our reference model is the standard textbook presentation [2]: a four-party path, per-hop fees, staggered CLTV timelocks $3t/2t/t$, and a two-round “lock then release” commit.

Contributions. In brief: (1) a modular, fully machine-checked model of the two-party channel lifecycle and multi-hop HTLC routing with fees (81 lemmas); (2) a forwarding-replay hardening that replaces a global axiom with a structural linear-resource guard; (3) a machine-checked treatment of the wormhole attack, both reachable and quantified to the exact stolen fees, plus a grieving DoS and economic-soundness theorems; (4) an empirical assumption-minimality result (removing the intermediary-claim restriction yields a concrete counterexample); and (5) a map of the prover’s

tractability boundaries—the signing/nat/clock OOM triple, signed-message N -hop routing, and reusable channels.

Scope. We model integrity and safety under a Dolev–Yao adversary [6] who controls the network and may compromise long-term keys. We *do not* model payment privacy / unlinkability (a confidentiality property requiring onion routing and observational-equivalence proofs)—this is stated as an explicit exclusion, not silently omitted. Timing is treated relationally (orderings of timeout events) except where a concrete block-height model is used to *derive* the CLTV windows.

2 Architecture: a modular five-theory package

A single monolithic theory is intractable: combining Tamarin’s [5] signing equational theory, natural-number induction, and an unbounded block clock exhausts memory. Crucially this was *tested*, not assumed—any *two* of the three ingredients remain tractable; only the full triple OOM-kills the prover. The model is therefore split so that no single theory carries all three (Table 1).

File	Aspect	Lemmas	Builtins
<code>multihop.spthy</code>	lifecycle + routing + value/fees + atomicity + wormhole	43	hashing, signing, nat
<code>Modif.spthy</code>	N -hop generalisation (generic linear-fact forward)	25	hashing, signing, nat
<code>Clock.spthy</code>	block-clock lifecycle & staggered-CLTV claim windows	9	natural-numbers
<code>Cltv.spthy</code>	pure CLTV block arithmetic (delta positivity)	3	natural-numbers
<code>timeout.spthy</code>	early-timeout race (liveness assumption removed)	1	—
Total		81	

Table 1: The five theories. `multihop.spthy` combines signing + nat once the unbounded clock is dropped and timing is kept relational—which is why value/fee conservation could be merged into it. `Modif.spthy` reuses the same signing + nat combination for a generic, any-length forward; the three small timing theories isolate the unbounded block clock away from signing.

3 Methodology

All results were produced under a strict *test-first* discipline: a property is claimed only after Tamarin reports it, and any “plausible” existence result is re-checked with a tightened variant before being trusted (Section 7 shows this catching a false positive). Restrictions (global trace filters) are treated as *assumptions*, not proofs; a suspiciously short proof is a warning sign, and each restriction is either derived elsewhere or documented as a liveness boundary.

Per-lemma invocation (rather than one bulk `-prove`) is used on the large theory to avoid OOM:

```
LANG=C.utf8 LC_ALL=C.utf8 tamarin-prover multihop.spthy \
--heuristic=c --stop-on-trace=seqdfs --derivcheck-timeout=0 --prove=<Lemma>
```

The flag `-stop-on-trace=seqdfs` rescues the heavy existence witnesses but *hangs* the natural-number arithmetic theories, so it is applied per-file. The whole suite is driven by `run.py` (or the `Makefile`, which delegates to it).

4 Results: the core model

4.1 Multi-hop routing (multihop.spthy)

Figure 2 maps the routing, value, and attack lemmas onto the four-party flow; Figure 3 maps the channel-lifecycle lemmas onto a single Alice↔Bob channel. All 43 lemmas verify.

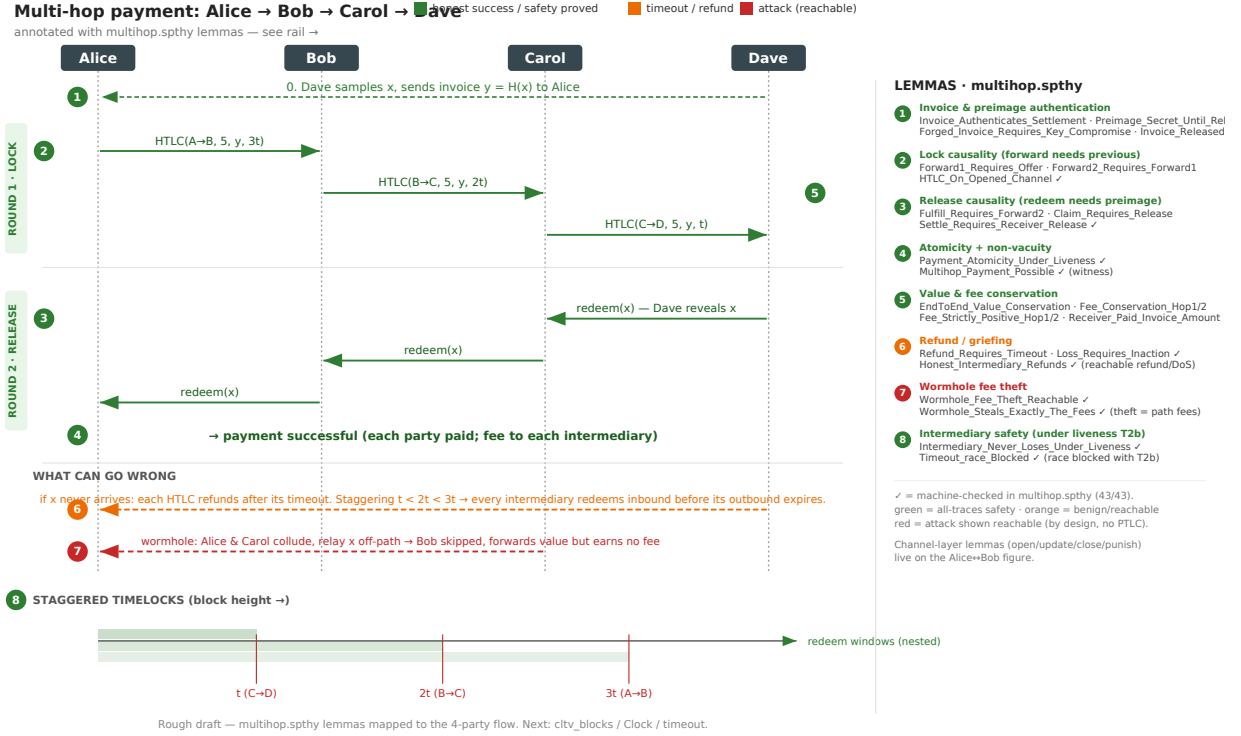


Figure 2: Four-party multi-hop payment. Round 1 locks HTLCs forward with staggered time-locks $3t/2t/t$; Round 2 releases the preimage backward. Numbered anchors 1–8 key to the multihop.spthy lemma rail. The red line is the wormhole; the amber line the timeout/refund unwind.

4.2 Timing safety (Cltv, Clock, timeout)

Figure 4 shows the staggered-timelock story, split across three small natural-number theories. *Cltv* derives the CLTV-delta invariant $d_{\text{out}} < d_{\text{in}}$ from concrete block numbers—i.e. the claim windows provably exist (CLTV_Gap_Is_Positive, Claim_Window_Nonempty, Staggered_Path_Safe). *Clock* runs a live block clock through the full forwarding lifecycle: it derives the claim window between adjacent hops (Claim_Window_Exists, Transitive_Preimage_Before_Upstream_Deadline), shows loss is exactly a skipped claim (Loss_Implies_Skipped_Claim), and adds lifecycle safety (No-HTLC_After_Close, Outcome_Exclusive) plus redeem/refund/ honest-flow reachability. *timeout* exhibits the early-timeout race that reappears once the liveness assumption T2b is removed (Early-Timeout_Race).

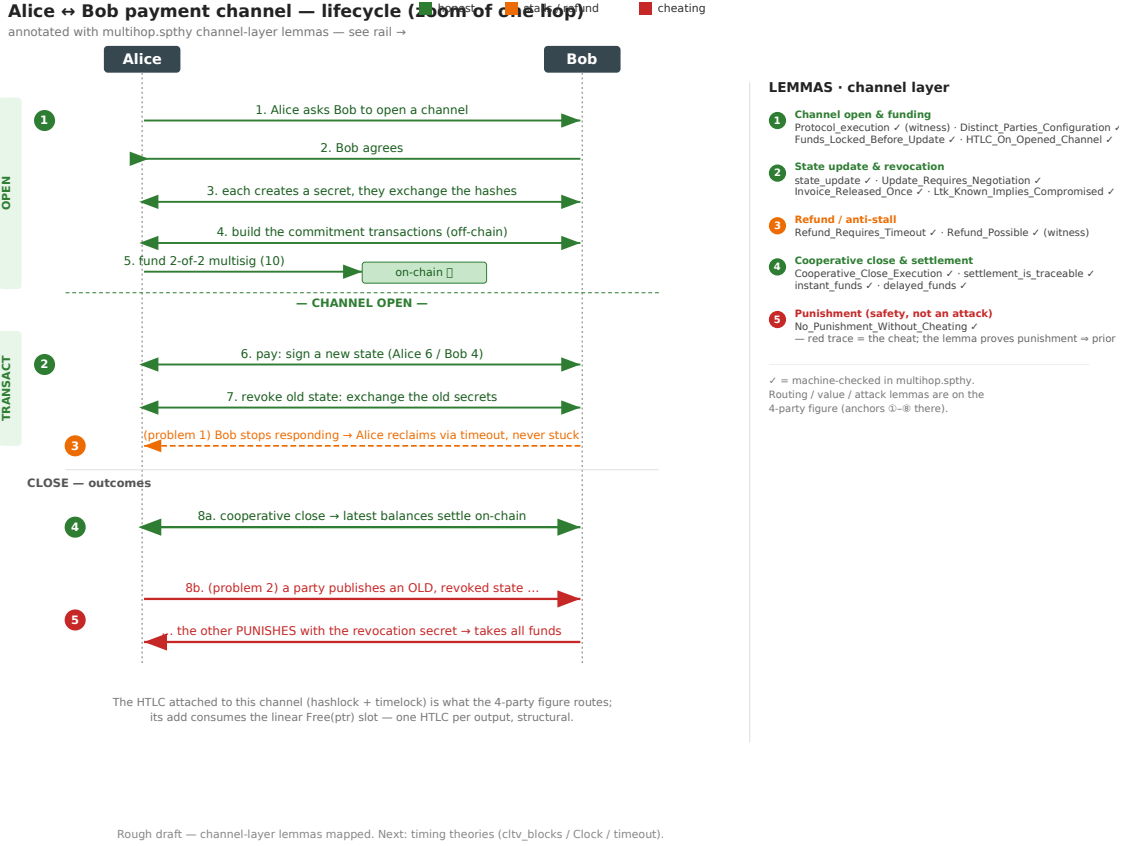


Figure 3: The Alice↔Bob channel lifecycle (a zoom of one hop): open, state update, refund, cooperative close, and punishment. Note anchor 5: the red trace is the *cheat*, but the lemma `No_Punishment_Without_Cheating` is a safety guarantee (punishment ⇒ prior cheat).

4.3 Statistics

5 Security analysis

5.1 Forwarding-replay hardening (axiom → structure)

The forwarding rules read the outgoing amount/fee from the network and are gated only by a public HTLC-offer message. A Dolev–Yao replay of that offer could re-fire an honest forward, locking a *second* HTLC on the same channel output with an adversary-chosen amount. This was found empirically via an audit probe (double-forward: reachable) and initially blocked with a global axiom `OneHTLCPerPtr`. We then replaced the axiom with a *structural* guard: each channel open emits one linear `Free(ptr)` token, and every HTLC-add consumes it. “At most one HTLC per output” is now a consequence of linear-fact consumption—one fewer assumption for the same guarantee. After the change the double-forward probe is falsified structurally (100 steps, no axiom) and all 43 lemmas still verify. We also confirmed that a symmetric “double-redeem” is *not* a vulnerability: the redeem/settle rules each consume a linear pending fact, so replay cannot double-spend an output.

Group	Class	Representative lemmas
Invoice / preimage auth	■	Invoice_Authenticates_Settlement, Preimage_Secret_Until_Released, Forged_ Invoice_Requires_Key_Compromise
Lock causality	■	Forward1_Requires_Offer, Forward2_ Requires_Forward1, HTLC_On_Opened_ Channel
Release causality	■	Fulfill_Requires_Forward2, Claim_ Requires_Release, Settle_Requires_ Receiver_Release
Atomicity / non-vacuity	■	Payment_Atomicity_Under_Liveness, Multihop_Payment_Possible
Value & fees	■	EndToEnd_Value_Conservation, Fee_ Conservation_Hop1/2, Fee_Strictly_ Positive_Hop1/2, Receiver_Paid_Invoice_ Amount
Refund / grieving	■	Refund_Requires_Timeout, Loss_Requires_ Inaction, Honest_Intermediary_Refunds
Wormhole	■	Wormhole_Fee_Theft_Reachable, Wormhole_ Steals_Exactly_The_Fees
Intermediary safety	■	Intermediary_Never_Loses_Under_ Liveness, Timeout_Race_Blocked
Channel lifecycle	■	state_update, Protocol_execution, Cooperative_Close_Execution, No_ Punishment_Without_Cheating, Funds_ Locked_Before_Update

Table 2: Lemma groups in `multihop.spthy` (43 total). Class: ■ safety proved, ■ benign/reachable, ■ attack reachable.

5.2 Wormhole attack (reachable and quantified)

The wormhole attack [3]: two colluding path members relay the preimage out-of-band, skipping an honest intermediary who forwards value but earns no fee. In the symbolic model the Dolev–Yao network *is* the colluders’ side channel. We machine-check it twice: `Wormhole_Fee_Theft_Reachable` (51 steps) exhibits a trace where the payment settles yet the middle hop is never redeemed and no intermediary earns a fee, with *no* key compromise; and `Wormhole_Steals_Exactly_The_Fees` (81 steps) quantifies the theft to precisely the path fees $\%f_1\% + \%f_2\%$, tying the attack to the value layer. This is the known limitation of the base HTLC construction (the fix in the literature is anonymous multi-hop locks / PTLCs, out of scope here); making it an explicit verified result turns “we did not model it” into “we machine-checked that the model exhibits it, and why.”

5.3 Economic soundness and grieving

Three all-traces theorems, none of which is assumed anywhere: `EndToEnd_Value_Conservation` (the sender locks the receiver’s amount plus a strictly positive total fee; no value created or destroyed); `Fee_Strictly_Positive_Hop1/2` (every hop’s fee is > 0 —structural, because the natural-number sort has no $\%0$, so a “free forward” is unreachable); and, as a reachability, `Honest_Intermediary_Refunds` (an honest intermediary induced to lock capital then refund on timeout, with no payment completing—the standard PCN grieving DoS).

Staggered CLTV timelocks — timing safety (block height →)

covers `cltv_blocks.spthy` · `Clock.spthy` · `timeout.spthy` — see rail →

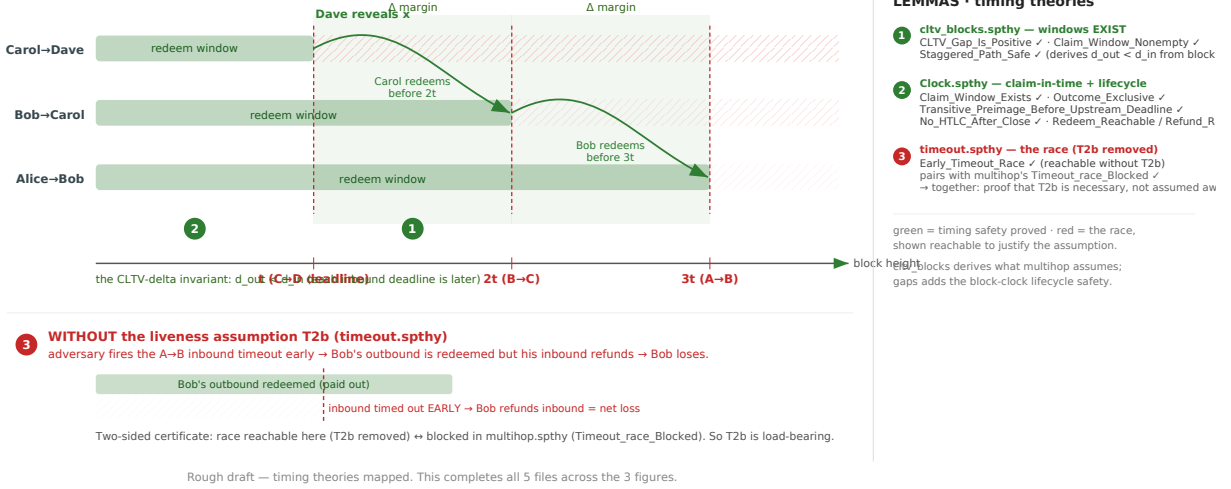


Figure 4: Staggered CLTV timelocks. The preimage propagates backward with a Δ margin at each hop because every inbound deadline is later than its outbound one. Bottom (red): with the liveness assumption T2b removed, the adversary fires the inbound timeout early and the honest intermediary suffers a net loss—`Early_Timeout_Race` in `timeout.spthy`. This pairs with `Timeout_Race_Blocked` in `multihop.spthy`, forming a two-sided certificate that T2b is necessary.

5.4 Assumption minimality (T3 is load-bearing)

A natural question is whether the intermediary-claim restriction T3 is redundant given T1 (redeem-before-timeout) and T2b (honest parties act before the inbound timeout). We tested it by removing T3 and re-running the suite: the safety lemma `Intermediary_Never_Loses_Under_Liveness` is then *falsified*, Tamarin finding a concrete 59-step counterexample. T2b orders the two timeouts but does not force the node to *sweep* the incoming HTLC once its outgoing one is redeemed; that action is exactly what T3 encodes. Hence the three timing restrictions are minimal: none is implied by the others.

6 Generalisation to N hops

The core model fixes the path at three hops. Replacing the two hardcoded forward rules with one *generic* rule was attempted in several encodings, all run:

The linear-fact route yields `Modif.spthy`, which carries amounts and fees (`Eq(%vin, %vout %+ %fee)`) and verifies **25 active lemmas in ≈ 103.5 s** (Tamarin 1.8, natural-numbers). A handful of `[use_induction]` causality lemmas (`Settle_Requires_Receiver_Release`, `Forward_Requires_Incoming`, `Claim_Requires_Release`, `Redeem_Requires_Receiver_Release`) are left commented as deliberate tractability boundaries: they key on the terminal settle and, combined with signing + natural-numbers, do not terminate on the generic chain, though they verify in the fixed-hop `multihop.spthy`. The framing is a sound proof layering: a concrete model with the full crypto adversary at three hops, and an abstraction with idealised linear-fact channels for routing safety at arbitrary N . The accompanying refinement theorem (`nhop_soundness`) shows the local per-hop

Lemma (representative)	Steps	Time	Kind
MultiHop_Payment_Possible	70	~77s	exists-trace (heaviest)
Forward2_Requires_Forward1	274	~47s	all-traces
Forward1_Requires_Offer	266	~46s	all-traces
Wormhole_Steals_Exactly_The_Fees	81	~45s	exists-trace
Wormhole_Fee_Theft_Reachable	51	~40s	exists-trace
Intermediary_Never_Loses_Under_Liveness	2	<1s	all-traces (under T2b)
EndToEnd_Value_Conservation	2	<1s	all-traces

Table 3: Representative per-lemma statistics for `multihop.spthy` (43 lemmas verify per-lemma sequentially in ≈ 16 min total on the test machine; average ≈ 26 s; many trivial lemmas finish in < 1 s).

Encoding	Outcome	
Signed-message generic forward	self-referential signed term; origin analysis re-	■
+ fresh per-hop id	curses, even a one-hop witness OOMs fixes replay but <i>not</i> termination—still a signed term producible only by a forward	■
Linear-fact routing (idealised channel)	safety invariants verify by induction over the chain, any N	■

Table 4: N -hop encodings. The bounding factor is cryptographic origin-analysis recursion on self-referential signed messages, not path length.

authentication proved in the concrete model transfers abstract safety to the concrete protocol for every N .

7 What failed, and the tractability boundaries

We record negative results explicitly—they are the paper’s methodological contribution as much as the positive ones.

- **The three-ingredient OOM.** Signing + natural-number induction + an unbounded block clock together exhaust the prover; any two are fine. This is the reason for the modular split (Section 2).
- **Signed-message N -hop routing** (Table 4): origin analysis on the self-referential ‘`htlc`’ term does not terminate, even for reachability, even with a fresh id.
- **A rejected restriction.** An initial fix `OneRedeemPerPtr` (“at most one redeem per output”) was tested and *rejected*: it falsified the honest witness `MultiHop_Payment_Possible`, because the two `Redeemed` events on the sender’s output in an honest run are the two endpoints’ views of one hop, not two spends. The correct guard was `OneHTLCPerPtr` (later made structural).
- **A caught false positive.** An exists-trace probe that shared only the payment *hash* between an offer and a payment appeared to show “value creation” (receiver paid more than the sender locked, no compromise). Tightening it to a single *linked* payment chain *falsified* it, while a control confirmed the chain is otherwise reachable—so the apparent attack was invoice-hash reuse across two unrelated offers, not a soundness bug. This is the adversarial-verification discipline in action.

- **Reusable channels (boundary).** We staged a refactor toward faithful reusable channels: Stage 1 (structural single-HTLC-per-output via the `Free(ptr)` slot) ships and re-verifies 43/43. Stage 2 (return `Free(ptr)` on resolution, so a channel carries sequential HTLCs) was spiked and is a clean tractability boundary: without `-auto-sources` every lemma returns “analysis incomplete” (the reuse loop leaves unbounded partial deconstructions); *with* `-auto-sources` the backward-reasoning safety lemmas verify (~ 45 s each), but every exists-trace witness and the interval lemma `No_Concurrent_HTLC_Per_Output` fail to converge (killed 500–700 s). Reusable channels are expressible and their core safety survives, but the loop reintroduces the witness/liveness non-termination the modular split avoids; Stage 1 is therefore the shipped model.

8 Reproduction

```
export LANG=C.utf8 LC_ALL=C.utf8
python3 run.py # full suite (5 theories)
python3 run.py multihop.spthy # one theory
python3 run.py Modif.spthy # the N-hop generalisation (25 lemmas)
```

Requires Tamarin 1.8 and Maude 3.1. The Makefile delegates to `run.py`, the single source of truth for per-file flags.

9 Conclusion

We delivered a modular, fully machine-checked model of Lightning multi-hop HTLC routing (81 lemmas over five theories), hardened it against a forwarding-replay flaw by turning a global axiom into a structural invariant, machine-checked the wormhole attack (reachable and quantified) and a grieving DoS, proved economic-soundness theorems, and established the minimality of the timing assumptions empirically. Equally, we mapped where the prover stops scaling: the three-ingredient OOM, signed-message N -hop routing, and reusable channels. The protocol-level results confirm expectations—consistent with the composable cryptographic treatment of Lightning [4], which our symbolic model complements at the routing layer; the durable contribution here is methodological—what it takes to keep Tamarin tractable on a stateful, time-locked, arithmetic protocol, with Lightning as the validating case study, and a discipline in which every claim (positive or negative) was executed rather than asserted.

References

- [1] J. Poon and T. Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. Technical report (draft 0.5.9.2), 2016.
- [2] M. Maffei. *Cryptocurrencies, Lecture 11: Scalability* (payment channels and payment-channel networks). TU Wien, 2024.
- [3] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei. *Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability*. In *NDSS*, 2019.
- [4] A. Kiayias and O. S. T. Litos. *A Composable Security Treatment of the Lightning Network*. In *IEEE CSF*, 2020.

- [5] S. Meier, B. Schmidt, C. Cremers, and D. Basin. *The TAMARIN Prover for the Symbolic Analysis of Security Protocols*. In *CAV*, 2013.
- [6] D. Dolev and A. C. Yao. *On the Security of Public Key Protocols*. *IEEE Transactions on Information Theory*, 29(2), 1983.

A Figure index

Figure 1 is the master map. Figures 2 and 3 carry `multihop.spthy`; Figure 4 carries the three timing theories (`Cltv`, `Clock`, `timeout`). All four are provided as PDF (embedded) and SVG (source) under `paper/figures/`. Colour key throughout: ■ honest/safety-proved, ■ benign failure, ■ attack reachable.